CO-1

24-12-2024

Data Structure :-

Data Structure name indicates itself Organising data in a memory.

- There are many ways of organising data in a memory i.e., contiguous memory

Ex:- Arrays in C

- Array is a collection of memory elements in which data is stored sequentially one after another.

- Data Structure is not a programming language like C, C++, Java. etc. It is a set of algorithms that we can use in any programming language to structure the data in memory.

Data Organisation :-

Organising data in particular order.

Management :-

It means managing the lots of data in a single place.

In DS we can do the data Organisation, management & store data that enables efficient

Ex:- Some companies have lots of data such as google, amazon, facebook etc. these all companies manage to store lots of data at a single place.

→ while storing the data it should be in a particular format so that it can be arised easily.

* Data structure is not a technology, not a software or not a data type.

Classification of Data structure

It is divided into two types.

Data Structure.

- linear data structure.
  - Static
    - Array ~~stack~~
  - Dynamic.
    - linked list
    - stack
    - Queue
- Non-linear data struc.
  - Trees
  - Graphs

'Linear Data Structures'

The elements are arranged linearly or sequentially where each element is attached to the previous or next adjacent elements.

Ex:- stack, Queue, Arrays.

Static :-

It is the memory size is fixed.

Ex:- Array.

Dynamic :-

The memory size can be fed during compile time and run time.

Ex:- Stack, Queue, linked list.

'Non-Linear Data Structure :'

The elements are not arranged in sequential order / non homogeneous memory

Ex:- Tree, Graph.

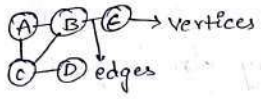
Tree :-

Every data structure consists of various nodes that are linked by edges.

. The structure of a tree is higherarchial that forms a relationships like Parent & child.

Graph :-

They come under NLDS which consists vertices and edges. The vertices are involved storing the data & edges show the relatio along the vertices.



Asymtotic Notations :-

This is used for analysis of algorithm

↓

we can evaluate the performance of algorithm in terms of input size.

- we can calculate the time & size of an algorithm by using asymtotic notations how much of time it will take.

- There are 3 types of Asymtotic notations.

(1) Big oh (O) Notation.

(2) Omega (Ω) "

(3) Theta (Θ) "

- There are 3 cases.

Best Case → Omega

Avg Case → Theta

Worst case → Big Oh.

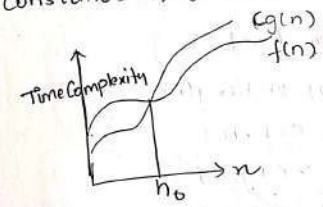
Big-Oh-Notations (O)

- This notation is the formal way to express the upper bound of an algorithm running tym. it measures worst case time Complexity.

Definition:-

let $f(n), g(n)$ be 2 non-negative function then $f(n)$ = order of $g(n)$

$\boxed{f(n) = Og(n)}$ if there exist 2 positive constants c, n_0 such as that $\boxed{f(n) \le c \times g(n)}$ $\forall n > n_0$.



c, n_0

$c = 1, n = 1$

$f(n) \le c * g(n)$

$2n+3 \le 1*n$

$2(1)+3 \le 1*1$

$5 \le 1$ (Not satisfied)

$c = 5, n = 1$

$2n+3 \le 5*n$

$2(1)+3 \le 5*1$

$5 \le 5$ (condition Satisfied

$c = 6, n = 1$

$2(1)+3 \le 6*1$

* The above condition is ~~time~~ true the constant va = 5.

Big Omega Notation (Ω)

→ This notation is a formal way to express lower bond of an algorithm time.

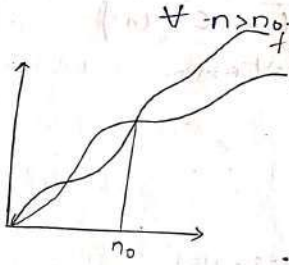
* It measures the best case time Complexity
* It takes a min amount of time.

Def^n:- It mainly represent 2 functions, $f(n)$, $g(n)$ be 2 non-~~neg~~ negative functions then $\boxed{f(n) = \Omega . g(n)}$

- If these exist 2 +ve c, n_0 such that

$$\boxed{f(n) \geq c * g(n)}$$

$\forall\ n > n_0$



Eg:- $f(n) = 2n+3$, $g(n) = 3$

$c = 1$, $n = 1$

$f(n) \geq c * g(n)$

$2n+3 \geq 1 * n$

$2(1)+3 \geq 1 * 1$

$5 \geq 1$ (condition satisfied)

- We can see that from above graph $f(n)$ & $g(n)$ functions have been Satisfied. When constant value is $= 1$

Theta Notation (Θ)

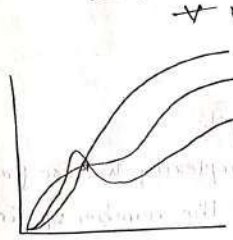
- This Notation Describes the formal way to express both lower bound & Upperbound of an algorithm running time.
- It measures avg. case running time.
- Def^n:- If $f(n)$, $g(n)$ be 2 non-Negative functions then.

$$\boxed{f(n) = \Theta . g(n)}$$

If there exist the constant c_1, c_2, n_0 such that

$$\boxed{c_1 . g(n) \leq f(n) \leq c_2 . g(n)}$$

$\forall\ n > n_0$



$f(n) = 3n+2$, $g(n) = n$

$c_1 = 1$, $c_2 = 4$.

$n = 1$ $\quad c_1 * g(n) \leq f(n) \leq c_2 . g(n)$

$1 \times n \leq 3n+2 \leq 4 . n$

$n \leq 3n+2 \leq 4n$.

$1 \leq 3+2 \leq 4$

$1 \leq 5 \leq 4$ [not Satisfied]

$n = 2$ $\quad c * g(n) \leq f(n) \leq c_2 . g(n)$

$1 \times n \leq 3n+2 \leq 4 . n$

$2 \leq 6+2 \leq 4 . 2$

$2 \leq 8 \leq 8$ [Satisfied]

Time Complexities [Performance of Algorithm]

- The amount of time that an algorithm requires for its execution is called time Complexity.

3 types of time Complexities.

1) Best Case :- Min amt of time for execution.
2) Worst Case :- Max amt of time for execution.
3) Avg Case :- Avg. amt of time for execution.

We have 2 approaches for calculating time Complexity.

1) Frequency Count / step Count.
2) Asymptotic Notation.

To Calculate Frequency Count / step Count :-

- In Order to Calculate time complexity we use frequency Count / step Count Method it Specifies the number of times a statement to be executed.

Rules to Calculate frequency Count :-

1) For Comments, declaration step count = 0
2) For Written and assigning the step count = 1
3) Ignore lower order exponents when higher order exponents are present.
4) Ignore Constants & Multiplies.

Ex 1 :- /* Sum of n natural numbers */

```
#include<stdio.h>
int main ){
  int sum=0, n,i; —1
  scanf("%d", &n); —1
  for(i=0; i<n; i++) {
      sum=sum+i;
  }
  Pf("%d", sum); —1
  return 0;
```

for(i=0; i<n; i++)

i=0 — 0<5 ✓
i=1 — 1<5 ✓
i=2 — 2<5 ✓
i=3 — 3<5 ✓
i=4 — 4<5 ✓
i=5 — 5<5 ✗

1+1+(n+1)+n+1+1
$\cancel{2}n+\cancel{5} \Rightarrow O(n)$
According to rules constants cahet

Eg 2 :- Identify Time Complexity for even (or) odd.

```
#include<stdio.h>
int main(){
    int n; —— 1
    scanf("%d",&n); — 1
    if(n%2==0) { — 1
        Printf("even"); — 1
    }
    else { —— 1
        Printf("odd"); — 1
    }
    return 0; — 1
}
```

for Constants Order is

1+1+1+1+1+1+1 = 7

$O(1)$

Eg 3 :- for(i=0; i<n; i++) — (n+1)

{ stmnts. — n

(n+1)+n

$= \cancel{2}n+\cancel{1} = O(n)$

Eg 4 :- for(i=n; i>0; i--) — n

{ stmnts ; — n.

}

$O(n)$

Eg5:- for(i=1; i<n; i+2) {
stmtnts; → O(n)
}.

Eg6:- for(i=0; i<n; i++) — n+1
{ statements; — n
}
for(j=0; j<n; j++) — n+1
{ statements; — n.
}
~~1~~n+~~1~~ → O(n)

Eg 7:- for(i=0; i<n; i++) — n+1
{
for(j=0; j<n; j++) — n*(n+1)
{ stmnts; — n
}
}

⇒ $n+1+n^2+n+n^2$
$= 2n^2+2n+1$
$= O(n^2)$

* for(i=1; i<n; i*2)
Eg8:- {
stmts;
}

Assume $i > n$
$2^k > n$
Apply log.
$\log_2 k = \log n$
$k = \log_2 n$ [cancel constant values]

i	n	i*2
1	1<10	1*2=2=2
2	2<10	2x2=4=2²
4	4<10	4x2=8=2³
8	8<10	8x2=16=2
⋮	⋮	
k	⋮	2^k.

Time Complexity :- $\boxed{O(\log n)}$

Eg 9:- for(i=0; i<n; i=i+2)
{
stmtnts;
}
n=10

iteration happend 5 times
$[\frac{n}{2}]$ times.

1. i=0 0<10 i=0+2=2
2. i=2 2<10 2+2=4
3. i=4 4<10 4+2=6
4. i=6 6<10 6+2=8
5. i=8 8<10 8+2=
i=10 10<10 ✗

Eg 10:- for(i=n; i>=1; i=i/2)
{
stmnts;
}

Assume $i<=1$

$\frac{n}{2^k} = 1$

$n = 2^k$

Apply log o.n b.s.

$\log n = \log 2^k$

$k = \log_2 n$

$i = n/2^2$,

$i = n/2^3$

$\vdots$

$i = n/2^k$.

Time Complexity $\boxed{O(\log n)}$

Eg 11:-

```
for(i=0; i<n; i++)
  {
    stmnts;
  }
for(j=0; j<m; j++)
  {
    stmnts;
  }.
```

$n+1+n$

$m+1+m$

$\cancel{2n} + \cancel{2m}$ (Remove coni

$\boxed{O(n+m)} \rightarrow$ T.C.

Eg 12:-

```
for(i=0; i<n; i++)          ——— n+1
{
  for(j=0; j<n; j=j*2)      ——— n*log n
  {
    stmnts;                 ——— n*n
  }
}
```

T.C $\rightarrow$ $\boxed{O(n\log n)}$ $\quad n+1+(n*\log n)+n^2$

Common Time Complexity:-

Big O Notation	Name of Big Oh Notation
$O(1)$	Constant
$O(n)$	linear.
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(\log n)$	logarithmic [Multiplication & Division. half the of the value]
$O(n\log n)$	$\rightarrow$ (Multiple loops with Multiplication & Division)
$O(\sqrt{n})$	Probability
$O(n!)$	Permutations.

Order of Time Complexity:-

$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \cdots 2^n < n$

Insertion Sorting :-

→ Insertion Sort is a Sorting algorithm that takes place all unsorted elements at its suitable place in each iteration.

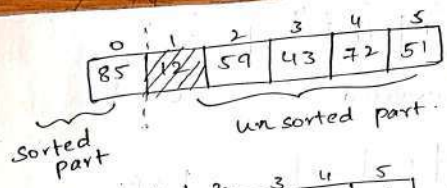
→ It is the Similar way of sorting the cards in our hands.

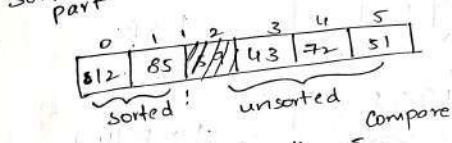
Eg:- We assume the sorted card a first card is sorted and remaining cards are unsorted order.

- We take 1 card from unsorted order & compare with the 1st card if the 1st card is greater than the unsorted card than we replace those cards in the same way all the cards will be Compare with sorted cards & arrange them in ascending order.

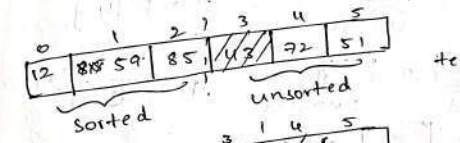
The elements in the insertion sort are placed in the array and vertually split into sorted & unsorted array.

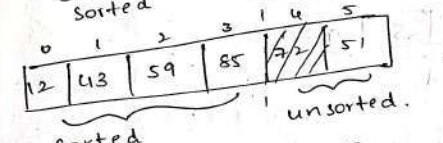
Values from the Unsorted part will be pick and placed at the Correct position in a Sorted order.

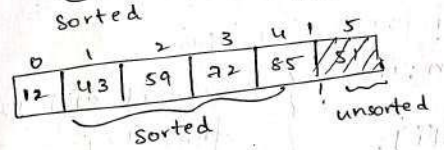


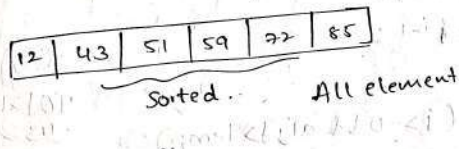


Compare with both 12 & 85 sorted part









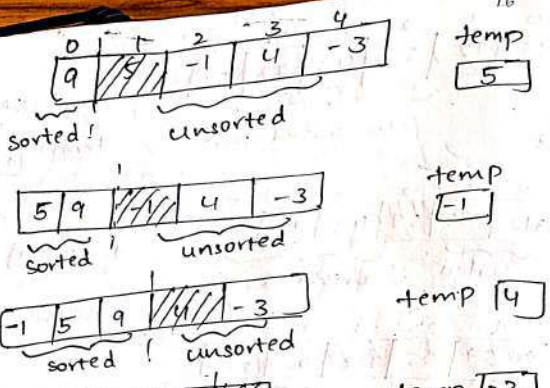
All elements are Sorted.

Time Complexity

Best Case :- $O(n)$

Avg Case :- $O(n^2)$

Worst Case :- $O(n^2)$



Logic

43 | 12 | 59

```
for(i=1; i<n; i++)
{
    temp=a[i];
    j=i-1;
}
for while (j>=0 && a[j]>temp)
{
    a[j+1]=a[j]
    j--;
}
a[j+1]=temp;
```

Tracing

temp = a[i] = a[1] = 12

temp = 12

j = i - 1

= 1 - 1 = 0 ⇒ j = 0

0 >= 0 && a[0] > temp

43 > 12

a[j+1]

0+1

a[1] = a[0]

43 = a[0]

j--; a[1]

a[0] = 12

Program

```
#include<stdio.h>
int main ()
{
int i, j, n, temp, a[25];
Printf ("Enter number of elements: "); // → 6
scanf ("%d", &n);
printf ("Enter %d elements : ", n); // → 80
for (i=0; i<n; i++)    //scan the array elements.
    scanf ("%d", &a[i]);
for (i=1; i<n; i++) //unsorted part loop
{
    temp = a[i];
    j= i-1;
while ((a[j]>temp)&&(j>=0))
//while (j>=0 && a[j]>temp)
{
    a[j+1]=a[j];
    j=j-1; // j--
}
a[j+1]=temp;
}
Printf ("Order of Sorted elements: ");
```

Input:- 6
80
18
45
-1
33
25

Output:-
Order of Sorted elements
-1 18 25 33 45 80

```
for (i=0; i<n; i++)
    printf("%d", a[i]);

return 0;
}
```

Shell Sort :-

Shell sort is an extension of insertion sort

- In insertion sort we move elements only one Position ahead when an element has to be moved far ahead many movements are involved.
- The idea of shells on is allowed. the elements are exchange of far items.
- In shells on we make the array h sorted for a large values of h.
- We keep reducing the values of h until it becomes one
- An array is said to be h sorted if all sublist of every h element are sorted

Ex:- 8, 6, 7, 2, 1, 4, 5, 3

0	1	2	3	4	5	6	7
8	6	7	2	1	4	5	3

↑i ↑j swap

Gap: No. of elements / 2 = 8/2 = 4

1	6	7	2	8	4	5	3

↑i ↑j swap — increment i & j

1	4	7	2	8	6	5	3

↑ ↑ swap

1	4	5	2	8	6	7	3

↑ swap

1	4	5	2	8	6	7	3

// pass ② ✓

Gap = Gap/2
= 4/2 = 2

1	4	5	2	8	6	7	3

↑i ↑j NO swap

1 4 5 2 8 6 7 3
↑i ↑j swap

1 2 5 4 8 6 7 3
↑i ↑j NO swap

1 2 5 4 8 6 7 3
↑i ↑j NO swap

1 2 5 4 8 6 7 3
↑i ↑j swap

1 2 5 4 7 6 8 3
↑i ↑j swap

1	2	5	4	7	3	8	6

↑i ↑j swap

Backward

1	2	5	3	7	4	8	6

//

Pass ②.

Gap = Gap/2 = 2/2 = 1

1 2 5 3 7 4 8 6
↑i ↑j NO swap

1 2 5 3 7 4 8 6
↑i ↑j NO swap

1 2 5 3 7 4 8 6
↑i ↑j swap

1 2 3 5 7 4 8 6
↑i ↑j NO swap

1 2 3 5 7 4 8 6
↑i ↑j swap

1 2 3 5 4 7 8 6
i j

1 2 3 4 5 7 8 6
NO swap

1 2 3 4 5 7 8 6
swap

1 2 3 4 5 7 6 8
i j

1	2	3	4	5	6	7	8

//

Pass ③

1	2	3	4	5	6	7	8
31	40	8	12	17	25	42	6

Gap = Gap/2 = 2/2 = 1

$= \frac{\text{No. of element}}{2} = \frac{9}{2} = 4.5$

$= 4$

2 3 4 5 6 7 8
40 8 12 17 25 42 6
↑j
swap 33>12

40 8 33 17 25 42 6
swap ↑j

40 8 33 31 25 42 6
↑i swap ↑j

25 8 33 31 40 42 6
↑i NO swap ↑j

25 8 33 31 40 42 6
↑i swap ↑j

25 8 6 31 40 42 33

✓

$/2 = 4/2 = 2$

2 3 4 5 6 7 8
25 8 6 31 40 42 33
↑j NO swap

25 8 6 31 40 42 33
↑j swap

25 17 6 31 40 42 33
↑i ↑j swap

6 17 25 31 40 42 33
↑i ↑j NO swap

6 17 25 31 40 42 33
i↑ NO swap ↑j

6 17 25 31 40 42 33
NO swap ↑i ↑j

6 17 25 31 40 42 33
i↑ swap ↑j

6 17 25 31 33 42 40

12 8 6 17 25 31 33 42
↑i ↑j swap

8 12 6 17 25 31 33 42
↑i ↑j swap

8 6 12 17 25 31 33 42
swap.

6 8 12 17 25 31 33 42
i↑ ↑j NO swap

6 8 12 17 25 31 33 42
i↑ ↑j NO swap

6 8 12 17 25 31 33 42
↑i ↑j NO swap

6 8 12 17 25 31 33 42
NO swap i↑ j↑

6 8 12 17 25 31 33 42
NO swap i↑ j↑

6 8 12 17 25 31 33 42
swap ↑i ↑j

6	8	12	17	25	31	33	40	42

Pass ③

Program :-

```
#include <stdio.h>
int main()
{
int n, i, j, t, gap, a[100];
Printf("enter n value: "); //9
scanf("%d", &n);
Printf("enter the elements");
for(i=0; i<n; i++)
  scanf("%d", &a[i]);
for(gap= n/2; gap>=1; gap=gap/2)
{
 for(j=gap; j<n; j++)
 {
  for(i=j-gap; i>=0; i=i-gap)
  {
   if(a[i+gap]>a[i])
   {
    break;
   }
   else
   {t=a[i];
    a[i]=a[i+gap];
    a[i+gap]=t;
   }
  }
 }
}
```

0 1 2 3 4 5 6 7 8
33 31 40 8 12 17 25 42 6

Gap = 8/2 = 4 ≥ 1 ✓

{ j=4; 4<8;

{ i=4-4=0; 0≥0

{ a[0+4]>a[0], i=i-gap = 0-4

12 > 33 ✗

break

else {

t=a[0] = 33

a[i]=12

a[i+gap]=33

}

```
Printf("the sorted array is: \n");
for(i=0; i<n; i++)
Printf("%d", a[i]);
return 0;
}
```

Time Complexity [shell sort]

Best Case :- $O(n \log n)$

Avg Case :- $O(n \log n)$

Worst Case :- $O(n^2)$

'Quick Sort'

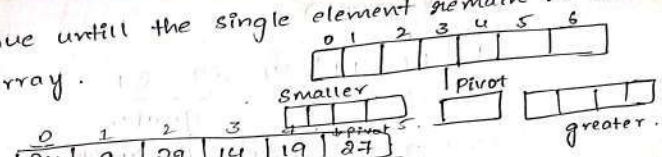
- Quick sort is a divide & conquer method approa
- Divide & conquer method is a technique of break ing down the algorithm into some Problems than solving the sub Problem & combining the sub Problem to get the result in a original order.
- the "pivot" element can be assigned as different ways
  - → First element can be Quicked as "pivot" & last element can be Quicked as "Pivot".
  - → Middle element can be Quicked as "Pivot"
  - Any element can be Quicked as "Pivot"

he key process of element is Quicked as Partition method.

uick sort picks an element as pivot and then partition the given array around the picked vot element.

k picks & In Quick sort a large array is divided to 2 arrays in which one poles holds values that e smaller than the specified value pivot and another array olds the values that are greater than pivot. ter that left and right sub arrays are also rtition using the same approach. It will

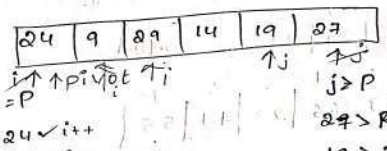
continue untill the single element remain in the sub-array.



pivot = 24 | i<=pivot, i++ | j> pivot, j--

i = 24
j = 27

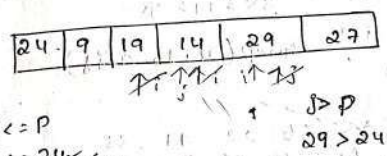
swap (a[i], a[j]) → Both conditions are false.



i<=P
24<=24 ✓ i++
9<=24 ✓ i++
29<=24 ✗

j>P
29>P24 ✓ j--
19>24 ✗

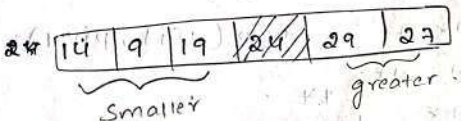
swap (a[i], a[j])



i<=P
19<=24 ✓ i++
14<=24 ✓ i++
29<=24 ✗

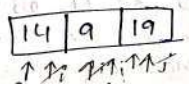
j>P
29>24 ✓ j--
14>24 ✗

When i cross j then swap (a[j], pivot)



Pivot is placed correct position

Apply Q.s



Pivot = 14
i<=P
9<=14 ✓
14<=14 ✓
19<=14 ✗

j>P
19>14 ✓
9>14 ✗

swap (a[j], pivot)

0	1	2	3
9	14	19	24

29 27 — greater

Apply Q.S.

Pivot = 9

29	27

p↑ i↑ ↑j

i<= P

~~swap~~ ~~swap~~

No elements have to increase i value so swap

29 ≤ 29 ✓

27 ≤ 29 ✓ if both conditions are true also

ping process apply Swap (a[j], pivot)

27	29

0	1	2	3	4	5
9	14	19	24	27	29

✓ Sorted.

Ex:-

0	1	2	3	4	5	6	7
77	11	88	33	66	22	44	55

p↑ i↑ i↑ ↑i ↑j

Pivot = 77

i = 77

j = 55

i<= P

77<= 77 ✓ i++

11<= 77 ✓ i++

88<= 77 ×

j > P

55

88 > 77 × j--

swap(a[i], a[j])

77 11 55 33 66 22 44 88

i↑ i↑ i↑ i↑ i↑j j↑i

i<= P

55<= 77 ✓ i++

33<= 77 ✓ i++

66<= 77 ✓ i++

22<= 77 ✓ i++

44<= 77 ✓ i++

88<= 77 ×

j > P

88 > 77 ✓ j--

44 > 77 ×

Swap (a[j], pivot)

~~77~~

Sorted

44	11	55	33	66	22	~~77~~	88

Smaller — Pivot — greater

Smaller → Apply Quick sort.

44 11 55 33 66 22

p↑ i↑ i↑ ↑j

Pivot = 44

i = 44

j = 22

i<= P

44<= 44 ✓ i++

11<= 44 ✓ i++

55<= 44 ×

j > P

22 > 44 ×

Swap (a[i], a[j])

44 11 22 33 66 55

i↑ i↑j i↑j ↑j↑i

i<= P

22<= 44 ✓ i++

33<= 44 ✓ i++

66<= 44 ×

j > P

66 > 44 × j--

55 > 44 ✓ j--

66 > 44 ✓ j--

33 > 44 ×

swap (a[j], pivot) ← j crosses i

33 11 22 | 44 | 66 55

i↑ — Smaller — pivot — greater.

33	11	22

p↑ i↑ i↑ ↑j

Pivot = 33

i = 33

j = 22

i<= P

33 <= 33 ✓ i++

11<= 33 ✓ i++

22<= 33 ✓ i++

Swap (a[j], pivot)

22 11 | 33 | 44

i<= pivot

22<= pivot ✓

11<= 22 ✓

swap (a[j], pivot)

~~33 11 22 44~~

11	22	33	44

Apply Quick sort Greater

66	55

Pivot=66 p↑ i↑ j↑

i=66 66 <= 66 ✓ swap (a[i], pivot)

j=55 55 <= 66 ✓

11	22	33	44	55	66	77	88

Final Sorting array.

Program :-

```
#include <stdio.h>
void quicksort (int a[30], int start, int end)
{
int i, j, pivot, temp;                    temp = a[i];
pivot = start;
  i = start;
  j = end;
while (i<j)
{
  while (i<j)
  {
    while (a[i]<= a[pivot])
    i++;
    while(a[j]>a[pivot])
    j--;
    if (i<j)
    {
      temp= a[i];
      a[i]= a[j];
      a[j]= temp;
    }
  }
  temp= a[pivot];
  a[pivot]=a[j];
  a[j]  =temp;
  quicksort (a, start, j-1);
  quick sort (a, j+1, end);
}
}
int main ()
{ int i, n, a[30];
  Printf ("enter n value:" ); //9
  Scanf ("%d", &n);
  printf ("Enter %d elements: ", n); //
  for (i=0; i<n; i++)
  Scanf ("%d", &a[i]);
  quicksort (a, 0, n-1);// index 0 to n-1 value.
  Printf ("Order of Sorted elements: ");
  for (i=0; i<n; i++)
  Printf ("%d", a[i]);
  return 0;
}
```

77
11
88
33
66
22
44
55
1

array

1 11 22 33 44 55 66 77 8

Quick Sort

Time Complexity

Best Case :- $O(n \log n)$

Avg Case :- $O(n \log n)$

Worst Case :- $O(n^2)$

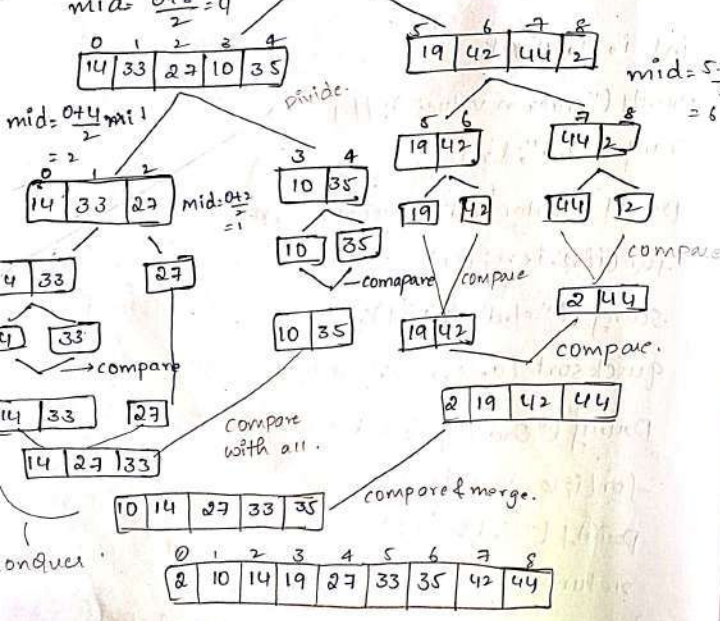
Merge Sort :-

- Merge sort is a Sorting technique based on divide & conquer method //combine
- First we are going to divide the list into sublist of two equal arrays. (keep on dividing the array into sub list until we get 1 element)
- After that we are going to merge the sublist until get 1 complete sublist that is sorted sublist/array

Ex:-

0	1	2	3	4	5	6	7	8
14	33	27	10	35	19	42	44	2

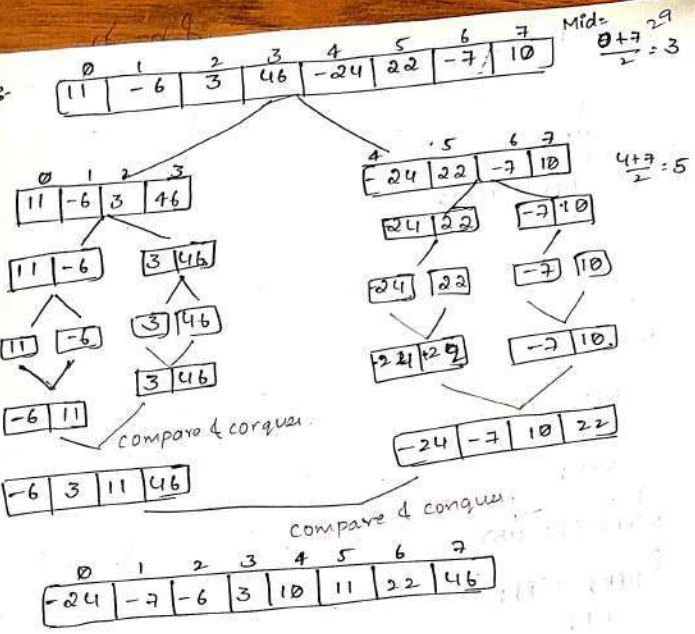
n=9



Ex 2 :-

0	1	2	3	4	5	6	7
11	-6	3	46	-24	22	-7	10

Mid= 0+7/2 = 3



Program :-

```
#include <stdio.h>
void merge (int a[], int lb, int mid, int ub, int b[])
{
  int i, j, k;
  i = lb;
  j = mid + 1;
  k = lb;
  while(i<= mid && j<= ub)
  {
    if (a[i] < a[j])
    {
```

lower bond

upper bond

```
            b[k]=a[i];
            i++;
        }
        else
        {
            b[k]=a[j];
            j++;
        }
        k++;
    }
    while (i<=mid)
    {
        b[k]=a[i];
        i++;
        k++;
    }
    while (j<=ub)
    {
        b[k]=a[j];
        j++;
        k++;
    }
    for(int k=lb;k<=ub;k++)
    {
        a[k]=b[k];
    }
}
void merge_sort(int a[], int lb, int ub, int b[])
{
    if (lb<ub)
    {
        int mid=(lb+ub)/2;
        merge_sort(a, lb, mid, b);
        merge(a, lb, mid, ub, b);
    }
}
int main() {
    int n;
    Printf("Enter the number of elements: ");
    scanf("%d",&n);
    int a[n], b[n];
    Printf("Enter the elements : ");
    for(int i=0; i<n; i++)
    {
        scanf("%d",&a[i]);
    }
    merge_sort(a,0,n-1,b);
    Printf("Sorted array: ");
    for(int i=0; i<n; i++)
    {
        printf("%d", a[i]);
    }
    Printf("\n");
    return 0;
}
```

Time Complexity.

Best Case :- $\Theta(n \log n)$

Avg Case :- $\Theta(n \log n)$

Worst Case :- $\Theta(n \log n)$

Ex:-

lb 0	1	2	3 ub
38	27	43	10

lb↓ 38 | 27 ↑mid — Parti-1

↓ub 43 | 10 ↑mid+1 — Parti-2

```
void mergesort (a, lb, ub)
{
  if (lb<ub)
    0 < 3
  {
    mid = (lb+ub)/2;
    mergesort (a, lb, mid)   // partition 1
    mergesort (a, mid+1, ub) // ,, 2
  merge (a, lb, mid, ub) //merging 2 partitions)
  }
}
```

'Bucket Sort' :- / Radix Sort :-

- Bucket Sort is a sorting technique that involves dividing elements into various groups or buckets ~~these buckets~~
- The Buckets are formed by uniformly distributing the elements.
- Once the elements are divided into buckets, they can be ~~using~~ sorted using any other sorting algorithm
- Finally the sorted elements are gathered together in an ordered ~~fashion~~ ~~passion~~ fashion.

Ex :-

0	1	2	3	4	4+1=5	6	7	8	9
15	1	321	10	802	2	123	90	109	11

We are placing 0 before the elements

015, 001, 321, 010, 802, 002, 123, 090, 109, 011

0: 010, 090.
1: 001, 321, 011
2: 802, 002,
3: 123,
4:
5: 015
6:
7:
8:
9: 109

Remove the elements. 010, 090, 001, 321, 011, 802, 002, 123, 015, 109

Pass 2 :-

0: 001, 802, 002, ~~015~~, 109
1: 010, 011, 015,
2: 321, 123,
3:
5:
6:
7:
8:
9: 090,

Remove all elements 001, 802, 002, 109, 010, 011, 015, 321, 123.

Pass 3:-

0: 001, 002, 010, 011, 015,	5:
1: 109, 123,	6:
2:	7:
3: 321	8: 802
4:	9:

Remove all E

0	1	2	3	4	5	6	7	8	9
253	10	1	16	1436	79	54	7	23	464

Ex:

0253 - 0010 0001 00016 1436 0079 00.54 .0007 0023

Pass 1:- (One's place)

0: 0010,	5:
1: 0001,	6: 0016, 1436,
2:	7: 0007
3: 0253, 0023	8:
4: 0054, 0464	9: 0079,

Remove all the elements.

Not sorted.

0010, 0001, 0253, 0023, 0054, 0464, 0016, 1436, 0007, 007

Pass 2:- (Tenth place)

0: 0001, 0007,	5: 0253, 0054,
1: 0010, 0016,	6: 0464
2: 0023	7: 0079
3: 1436,	8:
4:	9:

0001, 0007, 0010, 0016, 0023, 1436, 0253, 0054, 0464, 0079

Not sorted

Pass 3:- (Hundredth place)

0: 0001, 0007, 0010, 0016, 0023, 0054, 0079

1:

2: 0253,

3:

4: 1436, 0464,

Not Sorted

0001, 0007, 0010, 0016, 0023, 0054, 0079, 0253, 1436, 0464

Pass 4:- (Thousand's place)

0: 0001, 0007, 0010, 0016, 0023, 0054, 0079, 0253, 0464

1: 1436.

2:

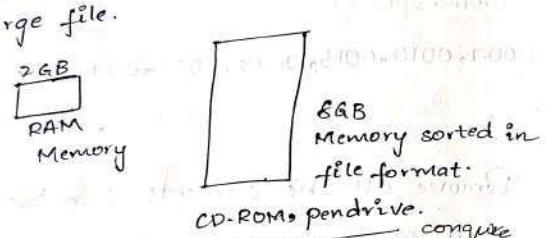
3: Remove all the elements from bucket

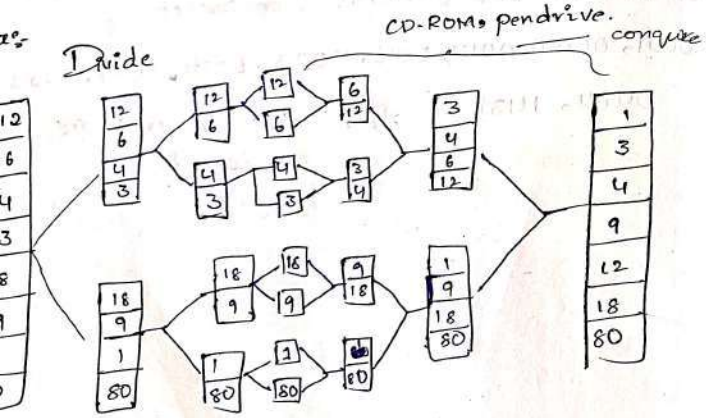
4: 0001, 0007, 0010, 0016, 0023, 0054, 0079, 0253, 0464, 1436

All the elements are Sorted.

External Sort :-

- It is applied on "external sort" which acquire large memory which cannot holded in main memory that is and is stored in Secondary memory that is hard disk.
- It uses the help of merge sort technique
- The data is divided into small chunks to fit into ram that can read, sort and also return out the temporary file during the sorting phase.
- In the merge phase the sorted sublist / subfiles are combine into single large file.





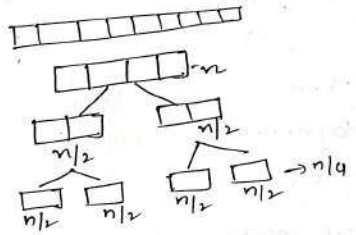
Time Complexity :-

Best Case :- $O(n \log n)$

Avg Case :- $O(n \log n)$

Worst Case :- $O(n \log n)$

Derivation of Time Complexity for Quick / Merge Sort Best Case :-



$T(n) = T(n/2) + T(n/2) + n$

$\boxed{T(n) = 2T(n/2) + n} \rightarrow (1)$

In n sub n/2

$T(n/2) = 2T\left(\frac{n/2}{2}\right) + n/2$

$T(n/2) = 2T(n/4) + n/2 \rightarrow (2)$

Sub. eq (2) in eq (1)

$T(n) = 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n \Rightarrow 4T\left(\frac{n}{4}\right) + \frac{2n}{2} + n$

$T(n) = 4T\left(\frac{n}{4}\right) + 2n$

$= 2^2 T\left(\frac{n}{2^k}\right) + 2n \rightarrow (3)$

$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + 3n \rightarrow (4)$

$\vdots$

$= 2^k T(n/2^k) + kn \rightarrow (5)$

Assume $\left[\frac{n}{2^k}\right] = 1$

~~$T(n) = 2^k T(1) + kn$~~

~~$n = 2^k$~~ $n = 2^k$

Apply log on both sides

$\log_2 n = \log_2 2^k$ $\quad [\because \log_a a = 1]$

$\boxed{\log_2 n = k}$ $\rightarrow (6)$

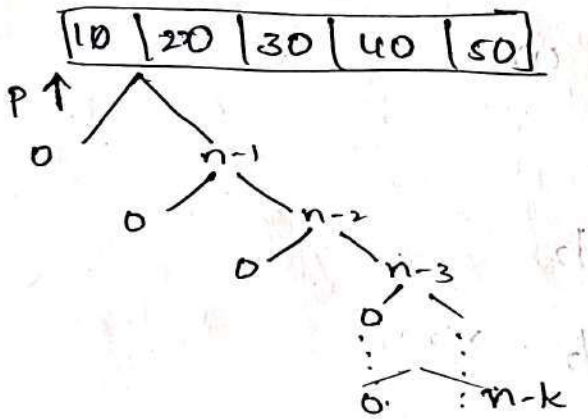
Subs eq(6) in eq(5)

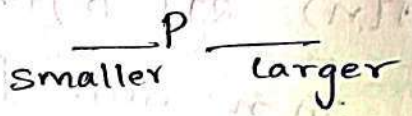
$T(n) = 2^k T\left(\frac{n}{2^k}\right) + k \cdot n$

$= n \cdot T(1) + \log n \cdot n$

$= n + n \cdot \log n$

Worst case of Quick Sort :- $O(n^2)$





$1+2+3+4+\dots n+(n-1)+(n-2)\dots$

Sum of natural numbers : $\frac{n(n+1)}{2} = \frac{n^2+n}{2} = \frac{n^2}{2} + \frac{n}{2} = O(n^2)$

This is greater constant = 1